

Practice 06 — IAM and Least Privilege

Objective: give a user exactly the permissions their job needs and **prove it**: show that what they're allowed to do works and that everything else fails. This is the skill from today's lecture.

Timebox: ~75 min of actual work. If you've been stuck on one step for more than 15 minutes, stop and ask. Don't push through alone.

How to submit: this practice is document-only. Nothing is autograded. Hand in a `REPORT.md` (or PDF) as your instructor tells you, with the **commands you ran**, their **output** (text or screenshots), and short answers to the questions at the end.

Pick your track

You don't need AWS access to pass. Pick **one** track:

	Track A — real AWS	Track B — no AWS
Needs	An AWS account or learner lab, AWS CLI v2	Python 3.9+, AWS CLI, ~5 min setup
Where policies run	Real IAM	moto , a local AWS emulator with permission checks turned on
Extra tasks	EC2 instance role, MFA, Policy Simulator	—

The IAM concepts are the same on both tracks:

AWS (Track A)	Track B equivalent
AWS account	Local moto server (<code>http://localhost:5002</code> , account <code>123456789012</code>)
Root user	The 3 "bootstrap" calls moto allows before it starts checking permissions
IAM users, groups, policies	The same IAM API calls, enforced by moto
Access keys / CLI profiles	Real <code>aws configure --profile</code> profiles, pointed at moto
EC2 instance role	<code>sts:AssumeRole</code> into a role from a CLI profile
Policy Simulator	You predict the result first, then run the request and compare
CloudTrail	moto's terminal log of every request

Definition of done

- ☐ An `admins` group exists with admin permissions, and your admin identity is a member. No daily work is done as root.
 - ☐ A `developers` group with a customer-managed policy `dev-s3-prefix` that allows reading and writing **only** `course-data/dev/*`.
 - ☐ A user `dana` in `developers`, with a CLI profile `dev`.
 - ☐ An evidence table with at least **6 requests** as `dana`: at least 2 allowed and 4 denied. Each row has your prediction, the actual result, and which rule decided it.
 - ☐ An explicit-deny test: `dana` is denied `dev/secrets/*` even though `dev/*` is allowed.
 - ☐ A role used through **temporary credentials** (`ASIA...`), not long-term keys.
 - ☐ Task 5 (policy debugging) solved and explained.
 - ☐ Cleanup done (Track A: required; Track B: stop the server).
-

Task 1 — Set up your environment

Track A — real AWS

1. Sign in as an admin identity, **not** root. If you only have root: turn on MFA for root, create a group `admins` with the AWS managed policy `AdministratorAccess`, create an IAM user for yourself in that group, and sign in as that user.
2. Configure the CLI for your admin identity and check who you are:

```
aws configure --profile admin          # or use your learner-lab credentials
aws sts get-caller-identity --profile admin
```

3. Create the bucket. Names are global, so add your student ID:

```
export BUCKET=course-data-<your-student-id>
aws s3 mb s3://$BUCKET --profile admin
echo "seed" > seed.txt && echo "hello" > hello.txt
aws s3 cp seed.txt s3://$BUCKET/prod/seed.txt --profile admin
```

In every policy below, replace `course-data` with your `$BUCKET` name.

Track B — no AWS (moto)

1. Install the tools. With `uv`:

```
uv tool install "moto[server]"
uv tool install awscli
```

Or use plain `pip install "moto[server]" awscli`. On Windows, run all of this inside WSL.

2. Start moto in a **separate terminal** and leave it running. `INITIAL_NO_AUTH_ACTION_COUNT=3` means the first 3 requests are free (your "root" moment) and every request after that is checked against IAM policies:

```
INITIAL_NO_AUTH_ACTION_COUNT=3 moto_server -p 5002
```

3. In your working terminal, point the CLI at moto and use the 3 free calls to create an admin. **Do exactly these three commands, in order.** Any extra request uses up a free call:

```
export AWS_ENDPOINT_URL=http://localhost:5002
export AWS_DEFAULT_REGION=us-east-1
export AWS_ACCESS_KEY_ID=bootstrap AWS_SECRET_ACCESS_KEY=bootstrap

cat > admin.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":"*","Resource":"*"}]}
EOF

aws iam create-user --user-name admin # free call 1
aws iam put-user-policy --user-name admin --policy-name admin-all \
    --policy-document file://admin.json # free call 2
aws iam create-access-key --user-name admin # free call 3 – copy the
keys!
```

4. Save the admin keys as a profile and stop using the bootstrap key:

```
unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY
aws configure --profile admin # paste AccessKeyId + SecretAccessKey, region us-east-1
aws sts get-caller-identity --profile admin
aws iam list-users --profile admin # works; the bootstrap key is now rejected
```

5. Create the admins group properly (as in Track A) and the bucket:

```
aws iam create-group --group-name admins --profile admin
aws iam put-group-policy --group-name admins --policy-name admin-all \
    --policy-document file://admin.json --profile admin
aws iam add-user-to-group --group-name admins --user-name admin --profile admin

export BUCKET=course-data
aws s3 mb s3://$BUCKET --profile admin
echo "seed" > seed.txt && echo "hello" > hello.txt
aws s3 cp seed.txt s3://$BUCKET/prod/seed.txt --profile admin
```

Keep `AWS_ENDPOINT_URL` set in every terminal you use for Track B, otherwise the CLI talks to real

Task 2 — A least-privilege developer (both tracks)

Save as `dev-policy.json` (replace `course-data` with your bucket on Track A):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListOnlyDevPrefix",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::course-data",
      "Condition": { "StringLike": { "s3:prefix": ["dev/*"] } }
    },
    {
      "Sid": "ReadWriteDevObjects",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::course-data/dev/*"
    }
  ]
}
```

```
ACCOUNT=$(aws sts get-caller-identity --profile admin --query Account --output text)

aws iam create-group --group-name developers --profile admin
aws iam create-policy --policy-name dev-s3-prefix \
  --policy-document file://dev-policy.json --profile admin
aws iam attach-group-policy --group-name developers \
  --policy-arn arn:aws:iam::$ACCOUNT:policy/dev-s3-prefix --profile admin

aws iam create-user --user-name dana --profile admin
aws iam add-user-to-group --group-name developers --user-name dana --profile admin
aws iam create-access-key --user-name dana --profile admin      # copy the keys
aws configure --profile dev                                     # paste dana's keys
aws sts get-caller-identity --profile dev                       # → .../user/dana
```

The policy is attached to the **group**, not the user. Explain why in your report.

Task 3 — Prove it: allowed vs denied

Before running each command, write down what you expect:  or . Then run it as `dana` (`--profile dev`) and fill in the table:

#	Request	Predicted	Actual	Decided by
1	<code>aws s3 cp hello.txt s3://\$BUCKET/dev/hello.txt</code>			
2	<code>aws s3 cp s3://\$BUCKET/dev/hello.txt -</code>			
3	<code>aws s3 cp hello.txt s3://\$BUCKET/prod/hello.txt</code>			
4	<code>aws s3 cp s3://\$BUCKET/prod/seed.txt -</code>			
5	<code>aws iam list-users</code>			
6	<code>aws ec2 describe-instances</code>			
7 (Track A only)	<code>aws s3 ls s3://\$BUCKET/dev/</code>			
8 (Track A only)	<code>aws s3 ls s3://\$BUCKET/</code>			

"Decided by" is one of: *Allow statement* `<Sid>` , *explicit deny*, or *implicit deny* (no matching allow).

Track B note: moto doesn't support S3 **listing** while permission checks are on (it fails with `SignatureDoesNotMatch` , which is a moto limitation, not your mistake). Skip rows 7–8. Every other row behaves the same as on real AWS.

Task 4 — Explicit deny and roles

4a. Explicit deny beats allow (both tracks)

Save as `deny-secrets.json` :

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": "s3:*",
    "Resource": "arn:aws:s3:::course-data/dev/secrets/*"
  }]
}
```

```
aws iam put-user-policy --user-name dana --policy-name deny-secrets \
  --policy-document file:///deny-secrets.json --profile admin

aws s3 cp hello.txt s3://$BUCKET/dev/secrets/db.env --profile dev    # predict first
aws s3 cp hello.txt s3://$BUCKET/dev/ok.txt --profile dev          # predict first
```

4b. Temporary credentials through a role

Track A — EC2 instance profile (no keys on the server):

1. Create a role `app-s3-reader` with trusted entity **EC2**, and attach an inline policy allowing only `s3:GetObject` on `arn:aws:s3:::<bucket>/prod/*`.
2. Launch a `t2.micro` / `t3.micro` Amazon Linux instance and attach the role under *Advanced details* → *IAM instance profile*. Connect with EC2 Instance Connect.
3. On the instance (don't run `aws configure`):

```
aws sts get-caller-identity          # → assumed-role/app-s3-reader/i-...
aws s3 cp s3://<bucket>/prod/seed.txt -    # works
aws s3 cp /etc/hostname s3://<bucket>/prod/x  # AccessDenied
ls ~/.aws                             # no credentials file anywhere
```

Track B — assume a role from a profile:

```

cat > trust.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"AWS":"arn:aws:iam::$ACCOUNT:user/dana"},"Action":"sts:AssumeRole"}]}
EOF
cat > reader.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Action":"s3:GetObject","Resource":"arn:aws:s3:::$BUCKET/prod/*"}]}
EOF

aws iam create-role --role-name prod-reader \
  --assume-role-policy-document file://trust.json --profile admin
aws iam put-role-policy --role-name prod-reader --policy-name read-prod \
  --policy-document file://reader.json --profile admin

# Try it – predict first:
aws sts assume-role --role-arn arn:aws:iam::$ACCOUNT:role/prod-reader \
  --role-session-name test --profile dev

```

That fails, because the trust policy lets dana assume the role but dana's **own** permissions don't include `sts:AssumeRole`. Fix it:

```

cat > allow-assume.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Action":"sts:AssumeRole","Resource":"arn:aws:iam::$ACCOUNT:role/prod-reader"}]}
EOF
aws iam put-user-policy --user-name dana --policy-name allow-assume \
  --policy-document file://allow-assume.json --profile admin

# A profile that assumes the role automatically, using dana's keys as the source:
aws configure set role_arn arn:aws:iam::$ACCOUNT:role/prod-reader --profile prod-reader
aws configure set source_profile dev --profile prod-reader
aws configure set region us-east-1 --profile prod-reader

aws sts get-caller-identity --profile prod-reader          # → assumed-role/prod-reader/...
aws s3 cp s3://$BUCKET/prod/seed.txt --profile prod-reader # works
aws s3 cp hello.txt s3://$BUCKET/prod/x --profile prod-reader # AccessDenied

```

Also run `aws sts assume-role ...` once by hand and include the output in your report, with the secret and token **redacted**. Point out the `ASIA...` key ID, the `SessionToken`, and the `Expiration`.

4c. Track A only — MFA and Policy Simulator

- Turn on a virtual MFA device for your own IAM user (and for root, if you haven't yet). Screenshot the *Security credentials* tab with the secret parts hidden.
- Open the **IAM Policy Simulator**, select user `dana`, and simulate `s3:GetObject` and `s3:PutObject` on `arn:aws:s3:::<bucket>/dev/x`, `.../prod/x` and `.../dev/secrets/x`.

Screenshot the results and check them against your Task 3 table.

Task 5 — Policy debugging (both tracks)

Each policy below was written by a teammate. For each one, say **what is wrong**, **what actually happens**, and give a **fixed** version. You can check your fix by attaching it to a new test user (on either track) and running the request.

5.1 — "Dana gets AccessDenied reading `reports/q1.csv`, but the policy clearly allows GetObject!"

```
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::course-data" }] }
```

5.2 — "The CI bot only needs to upload build artifacts."

```
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
    "Action": "s3:*",
    "Resource": "*" }] }
```

5.3 — "We denied deletes, but the intern still deleted a file."

```
{ "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": "s3:*",
      "Resource": "arn:aws:s3:::course-data/*" },
    { "Effect": "Deny", "Action": "s3:DeleteBucket",
      "Resource": "arn:aws:s3:::course-data/*" } ] }
```

5.4 — "Listing `dev/` works, but downloading anything from it fails."

```
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
    "Action": ["s3:ListBucket", "s3:GetObject"],
    "Resource": "arn:aws:s3:::course-data",
    "Condition": { "StringLike": { "s3:prefix": ["dev/*"] } } } ] }
```

Cleanup

Track A (required, to avoid charges and leftover access):

```
# terminate the EC2 instance from the console first
aws iam delete-user-policy --user-name dana --policy-name deny-secrets --profile admin
aws iam delete-user-policy --user-name dana --policy-name allow-assume --profile admin # if
created
aws iam remove-user-from-group --user-name dana --group-name developers --profile admin
aws iam list-access-keys --user-name dana --profile admin # then delete each:
aws iam delete-access-key --user-name dana --access-key-id <AKIA...> --profile admin
aws iam delete-user --user-name dana --profile admin
aws iam detach-group-policy --group-name developers \
    --policy-arn arn:aws:iam::$ACCOUNT:policy/dev-s3-prefix --profile admin
aws iam delete-group --group-name developers --profile admin
aws iam delete-policy --policy-arn arn:aws:iam::$ACCOUNT:policy/dev-s3-prefix --profile admin
aws s3 rb s3://$BUCKET --force --profile admin
```

Then delete the `app-s3-reader` role in the console, and remove the `dev` and `prod-reader` profiles from `~/.aws/credentials` and `~/.aws/config`.

Track B: press `Ctrl+C` in the moto terminal. Everything lives in memory and disappears. Remove the `admin`, `dev` and `prod-reader` profiles from `~/.aws/` so they don't get confused with real AWS profiles later.

Common mistakes

- **GetObject denied, but the policy "allows it".** Check the resource: object actions need `bucket/*` or `bucket/prefix/*`; bucket actions like `ListBucket` need the bare bucket ARN.
- **Track B: everything returns `InvalidClientTokenId`.** You used more than 3 requests before saving the admin key, or you're still exporting `AWS_ACCESS_KEY_ID=bootstrap`. Restart `moto_server` (it forgets everything) and redo Task 1 exactly.
- **Track B: commands suddenly hit real AWS.** You opened a new terminal and forgot `export AWS_ENDPOINT_URL=http://localhost:5002`.
- **assume-role denied even though the trust policy names the user.** Assuming a role needs **both** sides: the role's trust policy must trust you, and your own permissions must allow `sts:AssumeRole` on it.
- **Bucket already exists (Track A).** Bucket names are global across every AWS customer. Add your student ID.
- **Keys in your report.** Redact every `SecretAccessKey` and `SessionToken` before you submit, even the moto ones. Get used to doing it.

Report questions

1. Why is the developer policy attached to a group instead of the user?
2. In Task 4a, both an allow and a deny matched the same request. Which won, and what is the general rule?
3. Rows 5 and 6 in Task 3 were denied, yet no policy mentions IAM or EC2. Why?

4. What's the difference between an `AKIA...` key and an `ASIA...` key? Why is the second one safer to use on a server?
5. Name the AWS tool you'd use to (a) test a policy before deploying it, (b) find out who deleted an S3 object yesterday, (c) find buckets shared outside your account.
6. A teammate pushed an access key to a public GitHub repo, noticed, and force-pushed a commit that removes it. Is the problem solved? What must happen first?